
CHAPTER 2

Thoretical background

2.1 Artificial Neural Networks

Artificial Neural Networks (ANNs) are a type of computational model inspired by the structure and function of the human brain. ANNs consist of a large number of interconnected processing nodes or "neurons" that work together to solve complex problems.

These networks are able to learn and generalize patterns from data, making them a powerful tool for solving a wide range of problems, including classification, regression, prediction, and decision-making tasks. ANNs have been successfully applied in fields such as image and speech recognition, natural language processing, computer vision, robotics, and finance, among others.

ANNs are particularly well-suited for solving problems that are difficult to model using traditional statistical or algorithmic techniques, such as those involving non-linear relationships, complex interactions, or noisy or incomplete data. They can also be used for tasks where the underlying relationship between inputs and outputs is not well understood, and the goal is to learn a function that maps inputs to outputs based on examples provided in a training set.

There are many types of Artificial Neural Networks (ANNs), each with its unique architecture and application. Multi-Layer Perceptron (MLP) is one of the most popular types of ANNs, and it is widely used in various machine learning applications, such as in problems that consider tabular data.

2.1.1. Multi-layer Perceptron

There are many types of Artificial Neural Networks (ANNs), each with its unique architecture and application. Multi-Layer Perceptron (MLP) is one of the most popular types of ANNs, as it is a powerful tool for tabular data processing and they are also commonly used in applications such as classification, regression, and time series prediction, among others.

An MLP (Figure 2.1) is a type of feedforward neural network consisting of multiple layers (l) of neurons. The first layer is the input layer (x_n), which receives the input data. The output layer (y) produces the final output, while the layers between the input and output layers are called hidden layers. Each neuron in the input layer connects to each

neuron in the first hidden layer, and each neuron in the hidden layer connects to every neuron in the following layer until the output layer is reached.

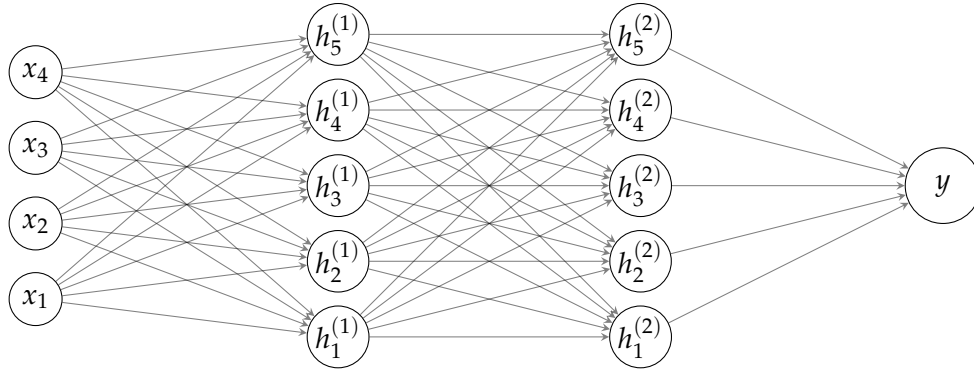


Figure 2.1: Architecture of a MLP with an input layer with 4 neurons, 2 hidden layers with 5 neurons each and the output layer with 1 neuron.

The connections between the neurons (n) in the MLP are weighted (h_n^l), and the network learns to adjust these weights during the training process. Each neuron in the MLP receives an input, applies a linear transformation to it, and passes it through a non-linear activation function such as the sigmoid or ReLU. This activation function introduces non-linearity into the network. If MLPs used linear activation functions, the output of the network would be a linear combination of the input features, regardless of the number of layers in the network. This would limit the network's ability to capture complex and non-linear relationships between the input and output variables. Moreover, the non-

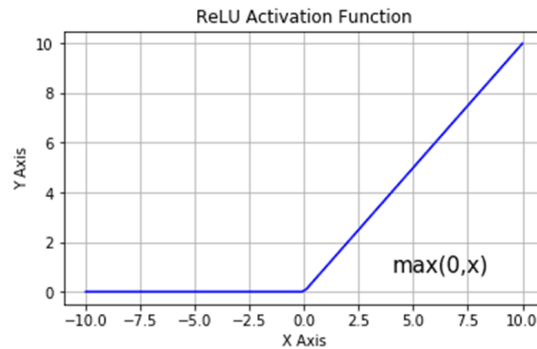


Figure 2.2: ReLU non-linear activation function.

linear activation functions introduce the property of "universal approximation" in MLPs. This means that MLPs with a single hidden layer and a sufficient number of neurons can approximate any continuous function to arbitrary accuracy. This property makes MLPs very powerful and effective for a wide range of machine learning problems.

2.1.2. Backpropagation

MLP neural networks are trained and optimized using a process called backpropagation. Backpropagation is an algorithm that uses the chain rule of calculus to calculate the gradients of the network's parameters with respect to a loss function, which measures the difference between the network's predicted output and the true output. The gradients are then used to update the network's parameters, such as the weights and biases of the neurons, using an optimization algorithm such as gradient descent. Gradient descent is a first-order optimization algorithm that minimizes the loss function by iteratively adjusting the parameters in the direction of the negative gradient.

During training, the output of the network is compared to the true output using the loss function. The gradients of the loss function with respect to the network's parameters are then calculated using backpropagation, and the parameters are updated using gradient descent. This process is repeated for multiple iterations, or epochs, until the network's performance on a validation set reaches a satisfactory level. In addition to gradient descent, other optimization algorithms, such as Adam and Stochastic Gradient Descent, can also be used to optimize the MLP's parameters. Stochastic Gradient Descent (SGD) and Adaptive Moment Estimation (Adam) are both optimization algorithms commonly used to train artificial neural networks, including multilayer perceptrons (MLPs).

Stochastic Gradient Descent is a variant of gradient descent that randomly selects a subset of the training data, known as a mini-batch, to calculate the gradient of the loss function. The gradients are then used to update the network's parameters, such as the weights and biases of the neurons. This process is repeated for multiple mini-batches until the entire training set has been processed, constituting one epoch. SGD is more computationally efficient than regular gradient descent and is less prone to getting stuck in local optima.

Adam is an adaptive optimization algorithm that combines ideas from both SGD and root-mean-square propagation (RMSprop). It computes adaptive learning rates for each parameter based on the historical gradients, allowing the algorithm to converge faster and more reliably than SGD. Adam also includes a bias correction term that corrects the bias introduced in the early stages of training when the estimated moments of the gradients are far from their true values. Adam is often used as a default optimization algorithm in many deep learning frameworks.

During the backpropagation phase of training Multi-layer Perceptron (MLP) neural networks, the gradients can suffer from two common problems: vanishing gradients and exploding gradients.

- **Vanishing gradients** occur when the gradients become very small, making it difficult to update the weights and biases of the lower layers of the network. This can cause the lower layers to become unresponsive to changes in the input, leading to poor performance. Vanishing gradients are often caused by the use of activation functions such as sigmoid or hyperbolic tangent, which have very small derivatives when their inputs are far from zero. To mitigate this problem, alternative activation functions, such as ReLU, have been developed that have larger derivatives and can prevent gradients from vanishing.
- **Exploding gradients**, on the other hand, occur when the gradients become very large, causing the weights and biases to be updated too drastically. This can lead to instability in the training process and cause the network to diverge instead of converging to a solution. Exploding gradients are often caused by numerical instability in the optimization algorithm or the network's architecture, such as very deep networks.

One of the phenomena that can cause one of these gradient problems is Internal Covariate Shift (ICS), where the distribution of the inputs to each layer of a multilayer perceptron (MLP) neural network changes during training, causing the network to converge more slowly. This change in the distribution of the inputs to each layer can create a mismatch between the statistics of the inputs to each layer and the statistics of the activations, which can slow down the convergence. Normalization techniques, such as Batch Normalization, can help mitigate ICS by normalizing the inputs to each layer to have zero

mean and unit variance before passing them to the activation function. This helps reduce the impact of ICS by making the statistics of the inputs to each layer more consistent and leading to more stable and efficient training of MLP neural networks.

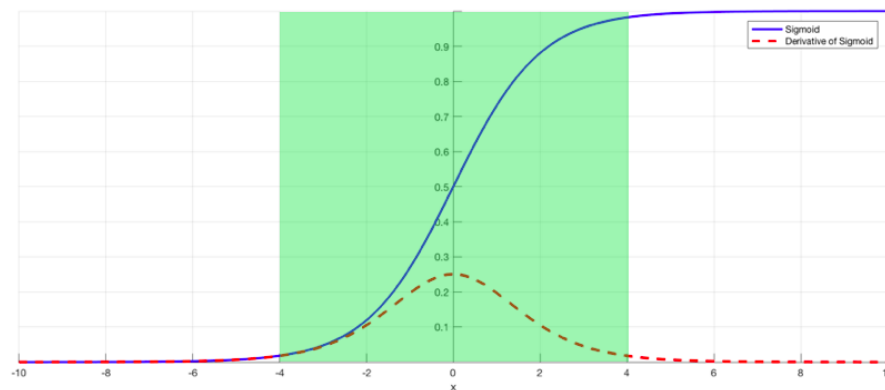


Figure 2.3: Sigmoid function and its derivative. We can see within the green box the non-saturate space of the function. Inputs outside this space will saturate the function and will bring the derivative to 0.

2.2 Gradient Boosting algorithms

In machine learning, a weak learner is a model that performs only slightly better than random guessing on a given task. While a weak learner may not be accurate on its own, it can be combined with other weak learners to form a strong learner through techniques such as boosting.

Gradient boosting algorithms are a family of machine learning algorithms that are designed to create predictive models by iteratively combining weak learners to form a strong learner. The basic idea of gradient boosting is to train each new model to correct the errors made by the previous models, using a gradient descent optimization process.

LightGBM is a gradient boosting algorithm that is designed to be both highly efficient and highly accurate. It uses a histogram-based approach to binning the data, which allows for faster training times and lower memory usage than other gradient boosting algorithms. Additionally, LightGBM uses a technique called Gradient-based One-Side Sampling (GOSS) to reduce the number of samples used in each iteration.

The basic idea of GOSS is to use a combination of gradient information and random sampling to identify and prioritize the most informative samples for each iteration of the algorithm. The training data is first sorted based on the absolute value of the gradient of the loss function with respect to the predicted value. Then, a small fraction of the samples with low gradients are randomly sampled, while the remaining samples with high gradients are used in each iteration. By doing this, GOSS prioritizes the samples that are more informative for the model while still including some randomly selected samples to prevent overfitting.

One of the key features of LightGBM is its ability to handle large datasets with high-dimensional features. It uses a technique called Leaf-wise Tree Growth, which grows the tree in a leaf-wise manner instead of level-wise, resulting in fewer leaf nodes and a

shallower tree. This can help to reduce overfitting and improve the speed of the training process

2.3 Underfitting vs Overfitting

Underfitting and overfitting are two common problems in machine learning that can affect the performance of models, including MLP and Gradient Boosting algorithms.

Underfitting occurs when a model is too simple to capture the underlying patterns in the data, resulting in poor performance on both the training and test data (Figure 2.4). In the case of MLP and Gradient Boosting algorithms, underfitting can occur when the model is too small or not trained for enough iterations, resulting in insufficient learning of the data patterns.

Overfitting occurs when a model is too complex and memorizes the training data instead of learning the underlying patterns, resulting in high performance on the training data but poor generalization to new data (Figure 2.4). In the case of MLP and Gradient Boosting algorithms, overfitting can occur when the model is too large or trained for too many iterations, resulting in excessive learning of the data patterns.

To address underfitting, one can increase the model's complexity, such as increasing the number of layers or trees in the case of MLP and Gradient Boosting algorithms, respectively. To address overfitting, one can reduce the model's complexity, such as reducing the number of layers or trees or applying regularization techniques like weight decay or dropout.

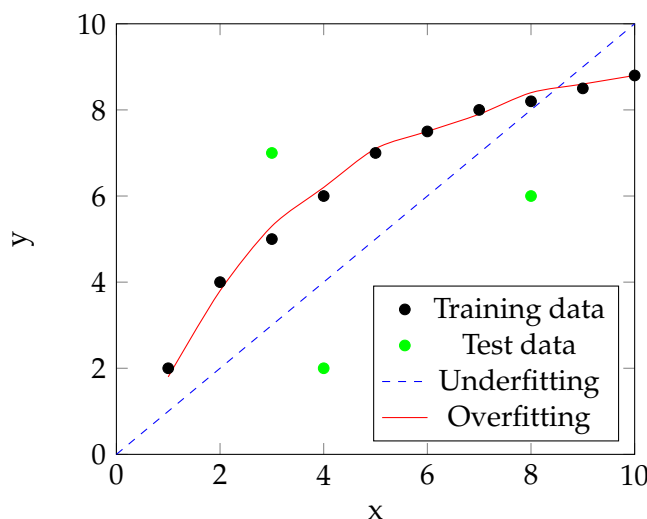


Figure 2.4: Visualization of underfitting and overfitting of a regression curve. The data points are the training data points.

Both underfitting and overfitting can be identified and mitigated through techniques like cross-validation, which involves splitting the data into training and validation sets to assess the model's performance on unseen data. Cross-validation is a technique used in machine learning to evaluate the performance of a model by testing it on a set of data that was not used during training. The idea behind cross-validation is to divide the available data into multiple subsets, called folds, and then train and test the model on each fold

while holding out the remaining folds for validation. Cross-validation can also help to find the optimal hyperparameters for the model, such as the number of layers, neurons, and trees, to balance the trade-off between underfitting and overfitting.